

Assignment 2: Policy Gradients

Due September 25, 11:59 pm

3 Policy Gradients

- Create two graphs:
 - In the first graph, compare the learning curves (average return vs. number of environment steps) for the experiments prefixed with `cartpole`. (The small batch experiments.)
 - In the second graph, compare the learning curves for the experiments prefixed with `cartpole_lb`. (The large batch experiments.)

For all plots in this assignment, the x -axis should be number of environment steps, logged as `Train_EnvstepsSoFar` (*not* number of policy gradient iterations).

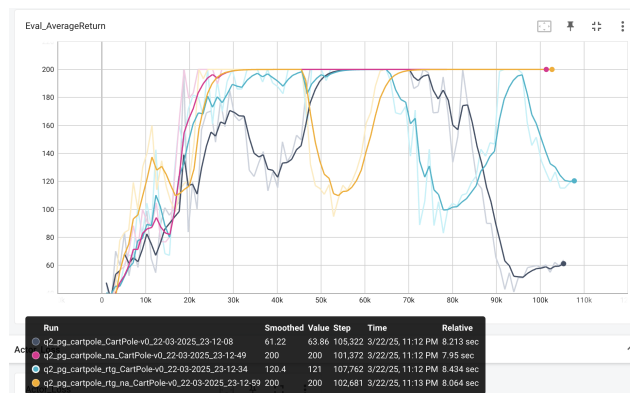


Figure 1: Learning curve for small batch cartpole experiments

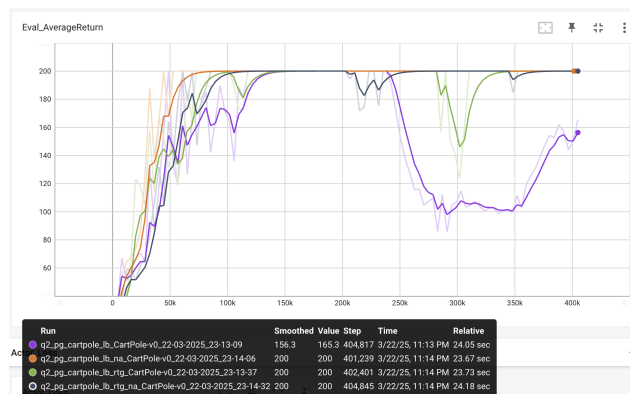


Figure 2: Learning curve for large batch cartpole experiments

- Answer the following questions briefly:
 - Which value estimator has better performance without advantage normalization: the trajectory-centric one, or the one using reward-to-go?
 - * The one using reward-to-go
 - Did advantage normalization help?
 - * Yes, all variants (trajectory centric v.s. reward-to-go, small v.s. large batch) benefited when advantage normalization is used.

- Did the batch size make an impact?
 - * Yes, training was more stable with a bigger batch size.
- Provide the exact command line configurations (or `#@params` settings in Colab) you used to run your experiments, including any parameters changed from their defaults.
 - No parameters were changed. The default command line configuration from the homework specs were used.

4 Neural Network Baseline

- Plot a learning curve for the baseline loss.

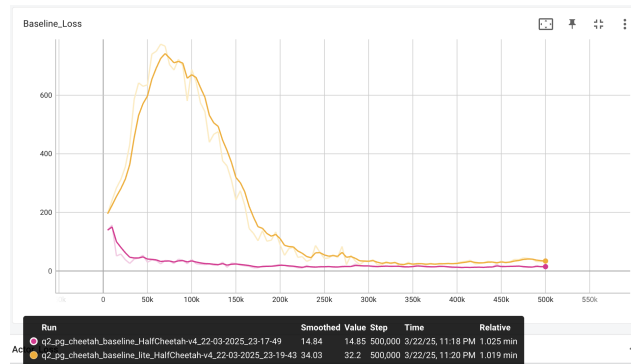


Figure 3: Learning curves for baseline losses

- Plot a learning curve for the eval return. You should expect to achieve an average return over 300 for the baselined version.

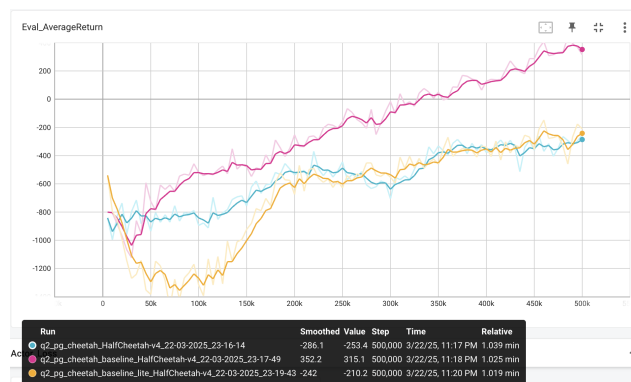


Figure 4: Learning curves for average eval returns

- Run another experiment with a decreased number of baseline gradient steps (`-bgs`) and/or baseline learning rate (`-blr`). How does this affect (a) the baseline learning curve and (b) the performance of the policy?
 - The baseline loss shot up initially and stabilized at about 2.5x of original setting.
 - Performance was even worse than the experiment without baseline initially, and eventually they roughly match.
 - Experiment setup: `lr=1e-3` (v.s. original `1e-2`) and `bgs=2` (v.s. original `5`)
- Optional:** Add `-na` back to see how much it improves things. Also, set `video_log_freq 10`, then open TensorBoard and go to the “Images” tab to see some videos of your HalfCheetah walking along!

5 Generalized Advantage Estimation

- Provide a single plot with the learning curves for the **LunarLander-v2** experiments that you tried. Describe in words how λ affected task performance. The run with the best performance should achieve an average score close to 200 (180+).
 - For $\lambda \in \{0, 0.95\}$, performance improved initially to be able to achieve positive returns, but regressed as learning continued. This is likely due to the high bias of the gradient estimates.
 - For $\lambda \in \{0.99, 1\}$, performance kept improving and eventually the returns flatten out at above 100. This seems to suggest that although higher λ values lead to higher variance, the reduction in bias outweighs the increase in variance and ultimately leads to better performance.
 - $\lambda = 0.98$ strikes the best balance between bias/variance and achieves the best peak performance (although less stable than higher λ values).

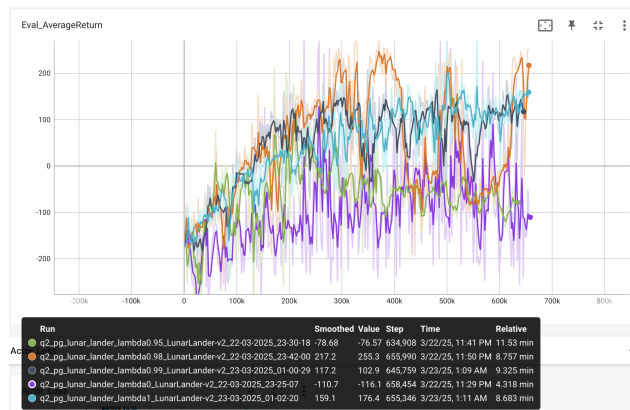


Figure 5: Learning curves for various λ values

- Consider the parameter λ . What does $\lambda = 0$ correspond to? What about $\lambda = 1$? Relate this to the task performance in **LunarLander-v2** in one or two sentences.
 - $\lambda = 0$ corresponds to 1-step Monte Carlo estimate (bootstrapping) of $A^\pi(s_t, a_t)$, whereas $\lambda = 1$ corresponds to full Monte Carlo estimate.
 - Lower λ values (0, 0.95) lead to higher bias lower variance, whereas higher λ values (0.99, 1) lead to lower bias and higher variance. In this particular setup, it seems to matter more to have lower bias than lower variance. $\lambda = 0.98$ is the best tradeoff and achieves the best performance.

6 Hyperparameter Tuning

1. Provide a set of hyperparameters that achieve high return on `InvertedPendulum-v4` in as few environment steps as possible.

```
for seed in $(seq 1 5); do
  python cs285/scripts/run_hw2.py --env_name InvertedPendulum-v4 -n 100 \
  --exp_name pendulum_best \
  --discount 0.95 -rtg --use_baseline -na --gae_lambda 0.98 \
  --batch_size 200 -l 3 -blr 1e-4 -bgs 1000 \
  --seed $seed
done
```

2. Show learning curves for the average returns with your hyperparameters and with the default settings, with environment steps on the x -axis. Returns should be averaged over 5 seeds.

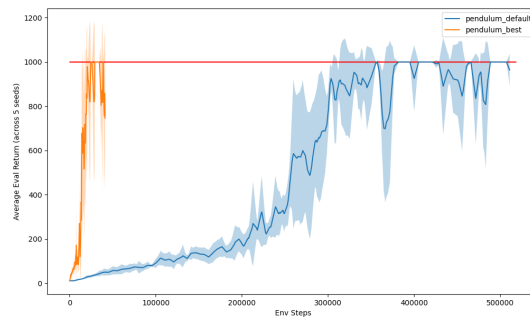


Figure 6: Learning curves for the best performing hyperparameters v.s. default

7 (Extra Credit) Humanoid

1. Plot a learning curve for the Humanoid-v4 environment. You should expect to achieve an average return of at least 600 by the end of training. Discuss what changes, if any, you made to complete this problem (for example: optimizations to the original code, hyperparameter changes, algorithmic changes).

8 Analysis

Consider the following infinite-horizon MDP:

$$a_1 \curvearrowright s_1 \xrightarrow{a_2} s_F$$

At each step, the agent stays in state s_1 and receives reward 1 if it takes action a_1 , and receives reward 0 and terminates the episode otherwise. Parametrize the policy as stationary (not dependent on time) with a single parameter:

$$\pi_\theta(a_1|s_1) = \theta, \pi_\theta(a_2|s_1) = 1 - \theta$$

1. Applying policy gradients

- (a) Use policy gradients to compute the gradient of the expected return $R(\tau)$ with respect to the parameter θ . **Do not use discounting.**

Hint: to compute $\sum_{k=1}^{\infty} k\alpha^{k-1}$, you can write:

$$\sum_{k=1}^{\infty} k\alpha^{k-1} = \sum_{k=1}^{\infty} \frac{d}{d\alpha} \alpha^k = \frac{d}{d\alpha} \sum_{k=1}^{\infty} \alpha^k$$

Solution:

- (b) Compute the expected return of the policy $\mathbb{E}_{\tau \sim \pi_\theta} R(\tau)$ directly. Compute the gradient of this expression with respect to θ and verify that this matches the policy gradient.

Solution:

2. Compute the variance of the policy gradient in closed form and describe the properties of the variance with respect to θ . For what value(s) of θ is variance minimal? Maximal? (Once you have an exact expression for the variance you can eyeball the min/max).

Hint: Once you have it expressed as a sum of terms $P(\theta)/Q(\theta)$ where P and Q are polynomials, you can use a symbolic computing program (Mathematica, SymPy, etc) to simplify to a single rational expression.

Solution:

3. Apply return-to-go as an advantage estimator.

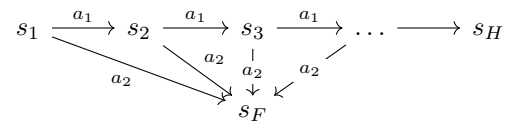
- (a) Write the modified policy gradient and confirm that it is unbiased.

Solution:

- (b) Compute the variance of the return-to-go policy gradient and plot it on $[0, 1]$ alongside the variance of the original estimator.

Solution:

4. Consider a finite-horizon H -step MDP with sparse reward:



The agent receives reward $R_{\max}$ if it arrives at s_H and reward 0 if it arrives at s_F (a terminal state). In other words, the return for a trajectory τ is given by:

$$R(\tau) = \begin{cases} 1 & \tau \text{ ends at } s_H \\ 0 & \tau \text{ ends at } s_F \end{cases}$$

Using the same policy parametrization as above, consider off-policy policy gradients via importance sampling. Assume we want to compute policy gradients for a policy π_θ with samples drawn from $\pi_{\theta'}$.

- Write the policy gradient with importance sampling.
- Compute its variance.

9 Survey

Please estimate, in minutes, for each problem, how much time you spent (a) writing code and (b) waiting for the results. This will help us calibrate the difficulty for future homeworks.

- **Policy Gradients:**
- **Neural Network Baseline:**
- **Generalized Advantage Estimation:**
- **Hyperparameters and Sample Efficiency:**
- **Humanoid:**
- **Humanoid:**
- **Analysis – applying policy gradients:**
- **Analysis – PG variance:**
- **Analysis – return-to-go:**
- **Analysis – importance sampling:**